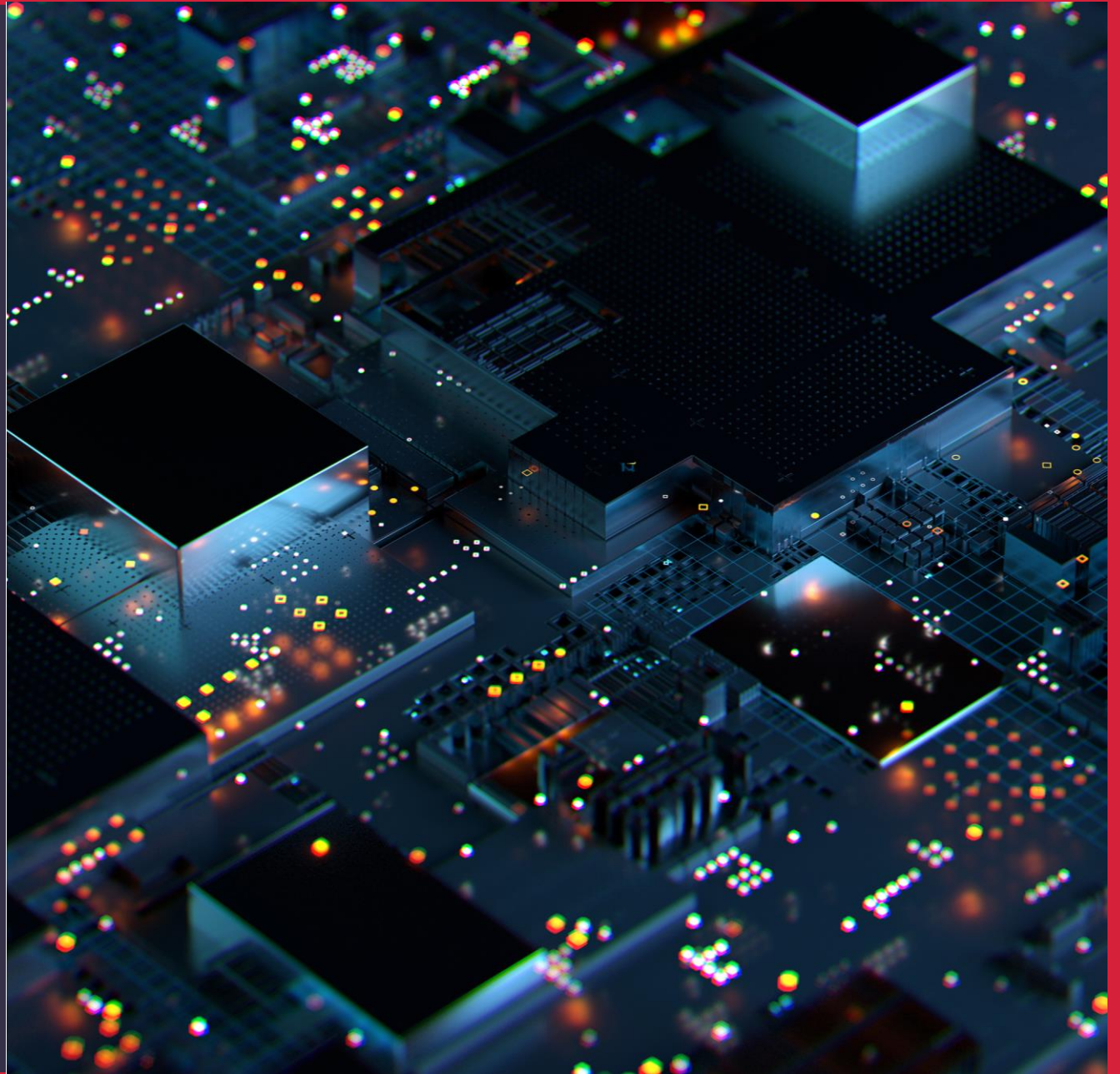


Infrastructure as Code - Provisioning Part 1

WELCOME TO CLASS 3!



Goals of the day



Create a VM in AWS, verify that it is running through the console



Use outputs to pipe useful information from provisioning to be used by CloudOps or Developers



Understanding use of modules in IaC practice to make code more reusable.



Photo by [David Becker](#) on [Unsplash](#)

©Maja Bjeletic@unsplash

Agenda

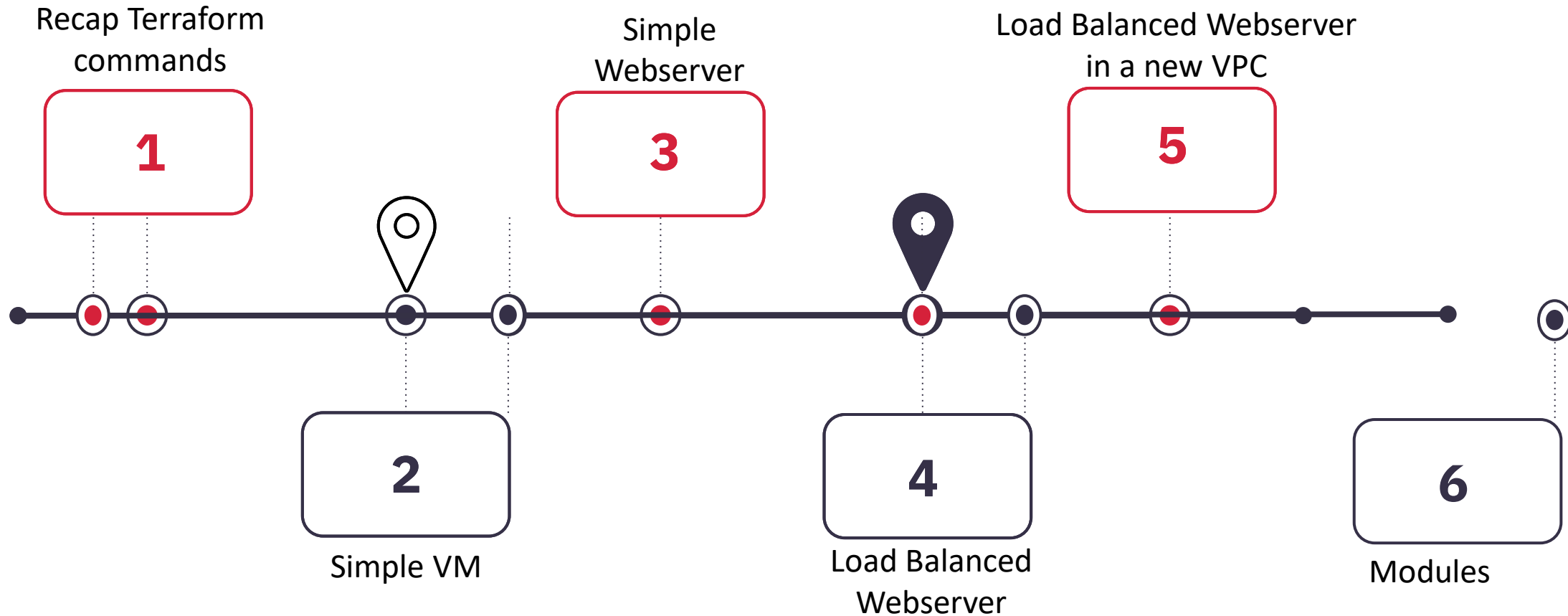
- Instructor led overview of Infrastructure as Code theory to be covered in this class
- Student review/run a Terraform implementation #1 to create a VM and verify that VM through AWS explorer
- **Break**
- Extend #1 to include additional AWS resources in Terraform 2
- Refactor #2 into modules as Terraform implementation #3
- Review modules as building blocks

Photo by David Becker on Unsplash

Review

- Did you complete your Terraform install OK?
- Why are we using Terraform here?
- Who owns it?
- Is it free?
- Brief Q & A

Journey Map



Infrastructure as Code Theory

- In your homework, you cloned very simple Terraform code to create an EC2 in AWS
- The EC2 doesn't do anything but you observed that it was created and that it exists
- You did all of this without logging into AWS, or did you ...
- What sorcery is this?



Photo by [Julius Drost](#) on [Unsplash](#)

Infrastructure as Code Theory (con't)

- We saw last week that Terraform is a clear leader in IaC
- Terraform uses APIs that supporting cloud providers create in order to manage infrastructure through code
- Terraform supports more than 3,000 integrations with more than 250 partners, including AWS, Microsoft Azure and Google Cloud
- Before looking at code implementations, let's review the most common commands used in Terraform, what they do and how they do it

terraform init

This is the first command you will use. It is safe to run many times. In attempting to initialize the working directory, it performs the following functions:

1. Assess backend configuration, defaulting to a local state file, but optionally can reference a Cloud storage location for state management. Detailed review of this topic later in the next class
2. Download and install provider plugins and load any specified provider credentials. Adds provider information (including versions) to dependency lock file, which should be committed to SCM
3. Retrieves source for modules that are included, either in an SCM or in the public Terraform Registry
4. Checks the code for any syntax errors and validates them

terraform plan

This is the second command you will use. By creating an execution plan, it allows you to preview the changes which this code will make to your infrastructure.

1. When run for the first time, it highlights how many elements will be created by the code
2. When run subsequently, it highlights how any changes to the code will modify the existing elements, or in the case where the infrastructure has changed but the code has not, it will highlight this drift between configuration and reality, making this command suitable for drift prevention
3. Can also run in refresh-only mode to ingest into the state any changes made to remote objects
4. Input variables can be provided to this command as a file or as individual variables
5. A plan file can be created that is then used by apply to create an exact configuration, despite any changes that may have been made to the infrastructure since the plan was created
6. This command is a subset of both `apply` and `destroy`

terraform apply

This is the third command you will use. It creates the infrastructure based on the configuration or plan provided.

1. Current state of infrastructure is compared with declared state and any update, deletes or creates needed are then applied to achieve this declared state
2. When a plan is provided, this command will implement without requesting any further approvals
3. Where no plan is provided, one is generated, and the user must approve before the requested infrastructure is provisioned. An auto-approve flag can override but should be used carefully
4. Output from apply can be codified to provide information on the infrastructure that was created, including server and EC2 details or application URLs

terraform destroy

This is the last command you will use. As the name suggests, it is used to remove infrastructure.

1. Within the scope of the current terraform project, all resources will be terminated
2. Like **apply**, the command invokes a **plan** detailing how the infrastructure will change and the user must confirm before any resource is destroyed
3. Don't forget to run this when finished labs or code reviews, otherwise you can run up significant cloud bills!!

Exercise 1: Simple VM (1 of 7)

Code Overview

- Lines 1-10 are configuration to support running Terraform
- Lines 12-14 indicate which AWS region will be used
- Lines 16-22 provide the details of what is required and how it is named

```
1  terraform {
2    required_version = ">= 0.15.0, < 2.0.0"
3
4    required_providers {
5      aws = {
6        source = "hashicorp/aws"
7        version = "~> 4.0"
8      }
9    }
10 }
11
12 provider "aws" {
13   region = "us-east-1"
14 }
15
16 resource "aws_instance" "firstEC2" {
17   ami           = "ami-0d6927ccef429da8c"
18   instance_type = "t2.micro"
19   tags = {
20     Name = "first_tf_EC2"
21   }
22 }
23
```

Exercise 1: Simple VM (2 of 7)

`terraform init`

- Nothing specified for the backend. AWS provider details provide and `init` is completed successfully

```
shane@Shanes-MacBook-Pro 01.CreateEC2 % terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v4.62.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
shane@Shanes-MacBook-Pro 01.CreateEC2 %
```


Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# aws_instance.firstEC2 will be created
+ resource "aws_instance" "firstEC2" {
  + ami                  = "ami-0d6927ccef429da8c"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "t2.micro"
  + ipv6_address_count     = (known after apply)
  + ipv6_addresses        = (known after apply)
  + key_name               = (known after apply)
  + monitoring             = (known after apply)
  + outpost_arn            = (known after apply)
  + password_data          = (known after apply)
  + placement_group        = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns            = (known after apply)
  + private_ip             = (known after apply)
  + public_dns             = (known after apply)
  + public_ip              = (known after apply)
  + secondary_private_ips  = (known after apply)
  + security_groups        = (known after apply)
  + source_dest_check      = true
  + subnet_id              = (known after apply)
  + tags                   = {
    + "Name" = "first_tf_EC2"
  }
  + tags_all              = {
    + "Name" = "first_tf_EC2"
  }
  + tenancy                = (known after apply)
  + user_data              = (known after apply)
  + user_data_base64       = (known after apply)
  + user_data_replace_on_change = false
  + vpc_security_group_ids = (known after apply)

  + capacity_reservation_specification {
    + capacity_reservation_preference = (known after apply)

    + capacity_reservation_target {
      + capacity_reservation_id          = (known after apply)
      + capacity_reservation_resource_group_arn = (known after apply)
    }
  }
}
```

Exercise 1: Simple VM (3 of 7)

terraform plan

- A lot is unknown when you run plan, just the items that you have configured are known at this point
- This gives you a view of what will change when you apply this terraform
- It will highlight syntax problems and errors
- Later we will use this command to detect changes in state or drift

Exercise 1: Simple VM (4 of 7)

terraform plan

- A lot is unknown when you run plan, just the items that you have configured are known at this point
- This gives you a view of what will change when you apply this terraform
- It will highlight syntax problems and errors
- Later we will use this command to detect changes in state or drift

```
+ ebs_block_device {
+   delete_on_termination = (known after apply)
+   device_name           = (known after apply)
+   encrypted             = (known after apply)
+   iops                  = (known after apply)
+   kms_key_id            = (known after apply)
+   snapshot_id           = (known after apply)
+   tags                  = (known after apply)
+   throughput            = (known after apply)
+   volume_id             = (known after apply)
+   volume_size           = (known after apply)
+   volume_type           = (known after apply)
+ }

+ enclave_options {
+   enabled = (known after apply)
+ }

+ ephemeral_block_device {
+   device_name = (known after apply)
+   no_device   = (known after apply)
+   virtual_name = (known after apply)
+ }

+ maintenance_options {
+   auto_recovery = (known after apply)
+ }

+ metadata_options {
+   http_endpoint           = (known after apply)
+   http_put_response_hop_limit = (known after apply)
+   http_tokens             = (known after apply)
+   instance_metadata_tags  = (known after apply)
+ }

+ network_interface {
+   delete_on_termination = (known after apply)
+   device_index          = (known after apply)
+   network_card_index    = (known after apply)
+   network_interface_id  = (known after apply)
+ }

+ private_dns_name_options {
+   enable_resource_name_dns_a_record  = (known after apply)
+   enable_resource_name_dns_aaaa_record = (known after apply)
+   hostname_type                      = (known after apply)
+ }

+ root_block_device {
+   delete_on_termination = (known after apply)
+   device_name           = (known after apply)
+   encrypted             = (known after apply)
+   iops                  = (known after apply)
+   kms_key_id            = (known after apply)
+   tags                  = (known after apply)
+   throughput            = (known after apply)
+   volume_id             = (known after apply)
+   volume_size           = (known after apply)
+   volume_type           = (known after apply)
+ }
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
shane@Shanes-MacBook-Pro 01>CreateEC2 %

Exercise 1: Simple VM (5 of 7)

`terraform apply`

- Repeats the `terraform plan` and asks if you wish to perform these actions
- Provides updates on progress with the resource creation, with the instance id of EC2 included in output

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.firstEC2: Creating...
aws_instance.firstEC2: Still creating... [10s elapsed]
aws_instance.firstEC2: Still creating... [20s elapsed]
aws_instance.firstEC2: Still creating... [30s elapsed]
aws_instance.firstEC2: Creation complete after 33s [id=i-0b461d2147c689c39]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
○ shane@Shanes-MacBook-Pro 01.CreateEC2 %
```

Exercise 1: Simple VM (6 of 7)

Instance state = running X Clear filters							
☐	Name ▾	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status	Availability Zone ▲
☐	first_tf_EC2	i-0b461d2147c689c39	✓ Running 🔍	t2.micro	✓ 2/2 checks passed	No alarms +	us-east-1a

```
resource "aws_instance" "firstEC2" {  
  ami = "ami-0d6927ccef429da8c"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "first_tf_EC2"  
  }  
}
```

terraform apply

Log into EC2 Dashboard to see the EC2 you created

AMI, Type & Name align

Default VPC and Subnet are used

Not possible to access the EC2, as no ports were opened.

It exists but does not do anything

Instance summary for i-0b461d2147c689c39 (first_tf_EC2) Info
Updated less than a minute ago

Instance ID i-0b461d2147c689c39 (first_tf_EC2)	Public IPv4 address 34.229.151.233 open address
IPv6 address -	Instance state ✓ Running
Hostname type IP name: ip-172-31-30-240.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-30-240.ec2.internal
Answer private resource DNS name -	Instance type t2.micro
Auto-assigned IP address 34.229.151.233 [Public IP]	VPC ID vpc-3150b84c
IAM Role -	Subnet ID subnet-1e523d53
IMDSv2 Optional	

Details Security Networking Storage Status checks Monitoring Tags

Instance details Info

Platform Amazon Linux (Inferred)	AMI ID ami-0d6927ccef429da8c
-------------------------------------	---------------------------------

⊗ **Failed to connect to your instance**
EC2 Instance Connect is unable to connect to your instance. Ensure your instance network settings are configured correctly for EC2 Instance Connect. For more information, see Set up EC2 Instance Connect at <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-connect-set-up.html>.

```

shane@Shanes-MacBook-Pro 01.CreateEC2 % terraform destroy
aws_instance.firstEC2: Refreshing state... [id=i-0b461d2147c689c39]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_instance.firstEC2 will be destroyed
- resource "aws_instance" "firstEC2" {
  ami           = "ami-0d6927ccef429da8c" -> null
  arn           = "arn:aws:ec2:us-east-1:799204530653:instance/i-0b461d2147c689c39" -> null
  associate_public_ip_address = true -> null
  availability_zone = "us-east-1a" -> null
  cpu_core_count   = 1 -> null
  cpu_threads_per_core = 1 -> null
  disable_api_stop = false -> null
  disable_api_termination = false -> null
  ebs_optimized    = false -> null
  get_password_data = false -> null
  hibernation      = false -> null
  id              = "i-0b461d2147c689c39" -> null
  instance_initiated_shutdown_behavior = "stop" -> null
  instance_state   = "running" -> null
  instance_type    = "t2.micro" -> null
  ipv6_address_count = 0 -> null
  ipv6_addresses   = [] -> null
  monitoring       = false -> null
  placement_partition_number = 0 -> null
  primary_network_interface_id = "eni-0bd140d4799e34d15" -> null
  private_dns      = "ip-172-31-30-240.ec2.internal" -> null
  private_ip       = "172.31.30.240" -> null
  public_dns       = "ec2-34-229-151-233.compute-1.amazonaws.com" -> null
  public_ip        = "34.229.151.233" -> null
  secondary_private_ips = [] -> null
  security_groups  = [
    "default",
  ] -> null
  source_dest_check = true -> null
  subnet_id        = "subnet-1e523d53" -> null
  tags             = {
    "Name" = "first_tf_EC2"
  } -> null
  tags_all         = {
    "Name" = "first_tf_EC2"
  } -> null
  tenancy          = "default" -> null
  user_data_replace_on_change = false -> null
  vpc_security_group_ids = [
    "sg-3087ed09",
  ] -> null

  capacity_reservation_specification {
    capacity_reservation_preference = "open" -> null
  }

  credit_specification {
    cpu_credits = "standard" -> null
  }

  enclave_options {
    enabled = false -> null
  }

  maintenance_options {
    auto_recovery = "default" -> null
  }

  metadata_options {
    http_endpoint         = "enabled" -> null
    http_put_response_hop_limit = 1 -> null
    http_tokens           = "optional" -> null
    instance_metadata_tags = "disabled" -> null
  }

  private_dns_name_options {
    enable_resource_name_dns_a_record = false -> null
    enable_resource_name_dns_aaaa_record = false -> null
    hostname_type                     = "ip-name" -> null
  }

  root_block_device {
    delete_on_termination = true -> null
    device_name           = "/dev/xvda" -> null
    encrypted             = false -> null
    iops                  = 100 -> null
    tags                  = {} -> null
    throughput            = 0 -> null
    volume_id             = "vol-012f727c48d51d5a5" -> null
    volume_size           = 8 -> null
    volume_type           = "gp2" -> null
  }
}

Plan: 0 to add, 0 to change, 1 to destroy.

```

Exercise 1: Simple VM (7 of 7)

terraform destroy

- Like the reverse of apply, it lets you know everything that will be removed and prompts you to confirm
- The instance ID is provided as a reference, with progress and summary included in output

```

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_instance.firstEC2: Destroying... [id=i-0b461d2147c689c39]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 10s elapsed]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 20s elapsed]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 30s elapsed]
aws_instance.firstEC2: Still destroying... [id=i-0b461d2147c689c39, 40s elapsed]
aws_instance.firstEC2: Destruction complete after 41s

Destroy complete! Resources: 1 destroyed.

```

Instance state = running X		Clear filters					
☐	Name ▾	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status	Availability Zone ▲
☐	first_tf_EC2	i-0b461d2147c689c39	🟢 Running 🔍	t2.micro	🟢 2/2 checks passed	No alarms +	us-east-1a

Exercise 2: Simple Webserver (1 of 11)

Code Overview

- Lines 1-22 are very similar to Exercise 1
- Lines 27, 38-56 specify a security group to be used by this EC2, which allows inbound traffic on port 80, but no outbound traffic
- Lines 29-35 through user data EC2 feature, runs a bash script to install and start Apache HTTPD

```
17 # Resource definition
18 resource "aws_instance" "webserver" {
19   ami           = "ami-0d6927ccef429da8c"
20   instance_type = "t2.micro"
21
22   tags = {
23     Name = "Web Server"
24   }
25
26   # Security group configuration
27   vpc_security_group_ids = ["${aws_security_group.webserver.id}"]
28
29   # User data for webserver configuration
30   user_data = <<-EOF
31   ...      #!/bin/bash
32   ...      sudo yum update -y
33   ...      sudo yum install httpd -y
34   ...      sudo service httpd start
35   ...      EOF
36 }
37
38 # Security group definition
39 resource "aws_security_group" "webserver" {
40   name        = "webserver"
41   description = "Allows HTTP traffic"
42
43   ingress {
44     from_port = 80
45     to_port   = 80
46     protocol  = "tcp"
47     cidr_blocks = ["0.0.0.0/0"]
48   }
49
50   egress {
51     from_port = 0
52     to_port   = 0
53     protocol  = "-1"
54     cidr_blocks = ["0.0.0.0/0"]
55   }
56 }
```


Exercise 2: Simple Webserver (2 of 11)

`terraform init`

- Nothing specified for the backend. AWS provider details provide and `init` is completed successfully.

```
shane@Shanes-MacBook-Pro 02.SimpleWebserver % terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v4.64.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
shane@Shanes-MacBook-Pro 02.SimpleWebserver %
```


Exercise 2: Simple Webserver (3 of 11)

terraform plan

```
# aws_instance.webserver will be created
+ resource "aws_instance" "webserver" {
+   ami           = "ami-0d6927ccef429da8c"
+   arn           = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone = (known after apply)
+   cpu_core_count = (known after apply)
+   cpu_threads_per_core = (known after apply)
+   disable_api_stop = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized = (known after apply)
+   get_password_data = false
+   host_id       = (known after apply)
+   host_resource_group_arn = (known after apply)
+   iam_instance_profile = (known after apply)
+   id           = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_state = (known after apply)
+   instance_type = "t2.micro"
+   ipv6_address_count = (known after apply)
+   ipv6_addresses = (known after apply)
+   key_name     = (known after apply)
+   monitoring   = (known after apply)
+   outpost_arn = (known after apply)
+   password_data = (known after apply)
+   placement_group = (known after apply)
+   placement_partition_number = (known after apply)
+   primary_network_interface_id = (known after apply)
+   private_dns   = (known after apply)
+   private_ip    = (known after apply)
+   public_dns    = (known after apply)
+   public_ip     = (known after apply)
+   secondary_private_ips = (known after apply)
+   security_groups = (known after apply)
+   source_dest_check = true
+   subnet_id      = (known after apply)
+   tags           = {
+     "Name" = "Web Server"
+   }
+   tags_all       = {
+     "Name" = "Web Server"
+   }
+   tenancy        = (known after apply)
+   user_data      = "471150ef31eaabc88158911e8a690f0aa1895290"
+   user_data_base64 = (known after apply)
+   user_data_replace_on_change = false
+   vpc_security_group_ids = (known after apply)
}
```

```
# aws_security_group.webserver will be created
+ resource "aws_security_group" "webserver" {
+   arn           = (known after apply)
+   description   = "Allows HTTP traffic"
+   egress        = [
+     {
+       cidr_blocks = [
+         "0.0.0.0/0",
+       ]
+       description = ""
+       from_port   = 0
+       ipv6_cidr_blocks = []
+       prefix_list_ids = []
+       protocol    = "-1"
+       security_groups = []
+       self        = false
+       to_port     = 0
+     },
+   ]
+   id           = (known after apply)
+   ingress      = [
+     {
+       cidr_blocks = [
+         "0.0.0.0/0",
+       ]
+       description = ""
+       from_port   = 80
+       ipv6_cidr_blocks = []
+       prefix_list_ids = []
+       protocol    = "tcp"
+       security_groups = []
+       self        = false
+       to_port     = 80
+     },
+   ]
+   name         = "webserver"
+   name_prefix  = (known after apply)
+   owner_id     = (known after apply)
+   revoke_rules_on_delete = false
+   tags_all     = (known after apply)
+   vpc_id       = (known after apply)
}
```

```
17 # Resource definition
18 resource "aws_instance" "webserver" {
19   ami           = "ami-0d6927ccef429da8c"
20   instance_type = "t2.micro"
21
22   tags = {
23     Name = "Web Server"
24   }
25
26   # Security group configuration
27   vpc_security_group_ids = ["${aws_security_group.webserver.id}"]
28
29   # User data for webserver configuration
30   user_data = <<-EOF
31   .....    #!/bin/bash
32   .....    sudo yum update -y
33   .....    sudo yum install httpd -y
34   .....    sudo service httpd start
35   .....    EOF
36 }
37
38 # Security group definition
39 resource "aws_security_group" "webserver" {
40   name           = "webserver"
41   description    = "Allows HTTP traffic"
42
43   ingress {
44     from_port = 80
45     to_port   = 80
46     protocol = "tcp"
47     cidr_blocks = ["0.0.0.0/0"]
48   }
49
50   egress {
51     from_port = 0
52     to_port   = 0
53     protocol = "-1"
54     cidr_blocks = ["0.0.0.0/0"]
55   }
56 }
```

Exercise 2: Simple Webserver (4 of 11)

`terraform apply`

- Repeats the `terraform plan` and asks if you wish to perform these actions
- Provides updates on progress with the resource creation, with the ids of both security group and EC2 instance included in output
- Security group created first as it is a dependency for the EC2

```
Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.webserver: Creating...
aws_security_group.webserver: Creation complete after 4s [id=sg-012d88d4e0c5d00ea]
aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [10s elapsed]
aws_instance.webserver: Still creating... [20s elapsed]
aws_instance.webserver: Still creating... [30s elapsed]
aws_instance.webserver: Creation complete after 32s [id=i-0ffb4b47bd571706a]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
shane@Shanes-MacBook-Pro 02.SimpleWebserver %
```

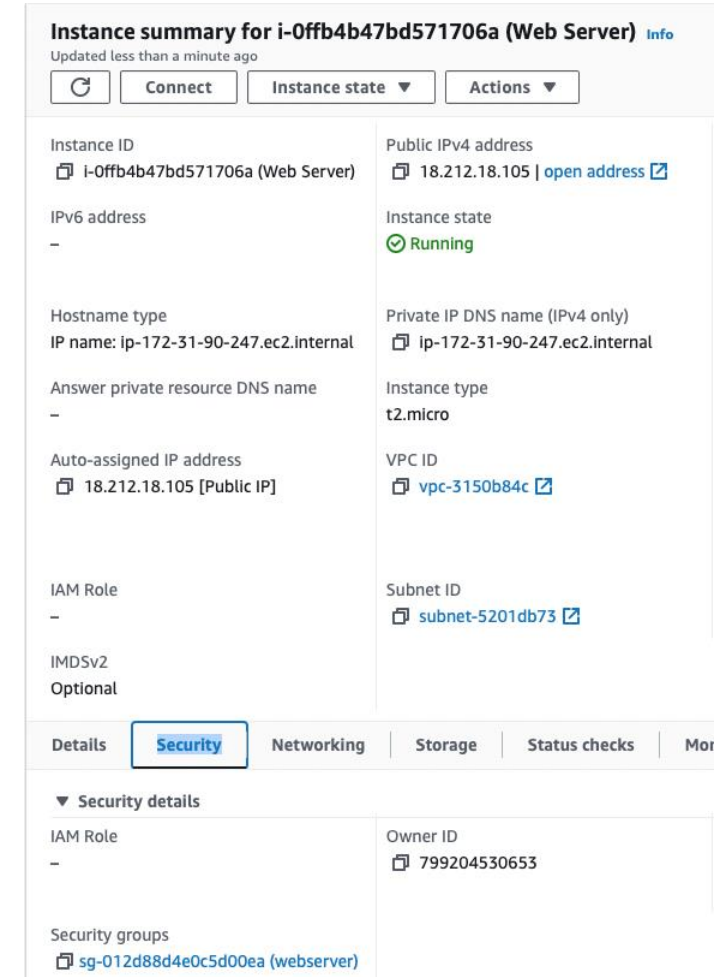
Exercise 2: Simple Webserver (5 of 11)

terraform apply

- Log into EC2 Dashboard to see the EC2 you created
- From console output

aws_security_group.webserver: id=sg-012d88d4e0c5d00ea

aws_instance.webserver: id=i-0ffb4b47bd571706a



Instance summary for i-0ffb4b47bd571706a (Web Server) [info](#)
Updated less than a minute ago

[Refresh](#) [Connect](#) [Instance state](#) [Actions](#)

Instance ID i-0ffb4b47bd571706a (Web Server)	Public IPv4 address 18.212.18.105 open address
IPv6 address -	Instance state Running
Hostname type IP name: ip-172-31-90-247.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-90-247.ec2.internal
Answer private resource DNS name -	Instance type t2.micro
Auto-assigned IP address 18.212.18.105 [Public IP]	VPC ID vpc-3150b84c
IAM Role -	Subnet ID subnet-5201db73
IMDSv2 Optional	

[Details](#) [Security](#) [Networking](#) [Storage](#) [Status checks](#) [More](#)

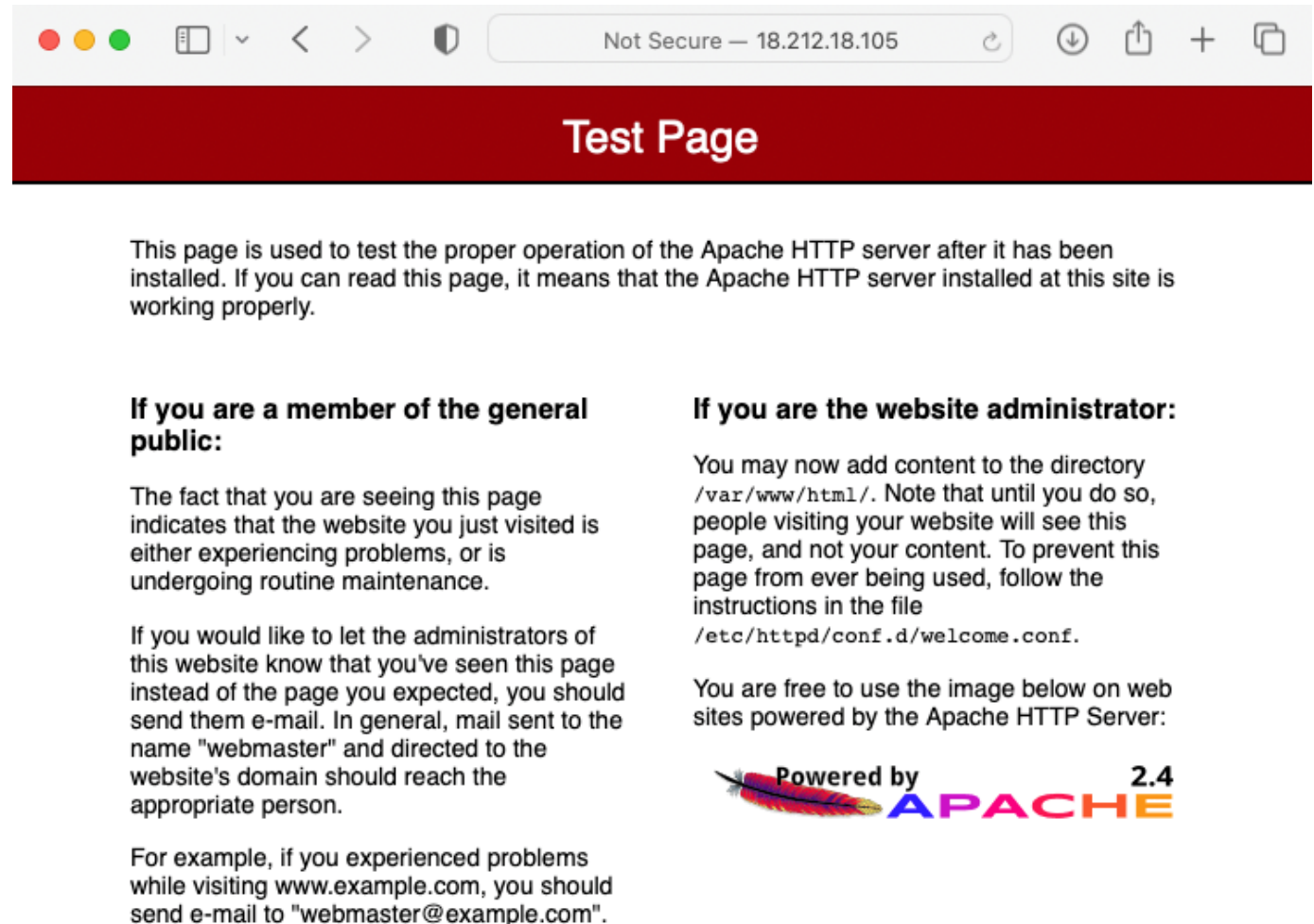
▼ **Security details**

IAM Role -	Owner ID 799204530653
Security groups sg-012d88d4e0c5d00ea (webserver)	

Exercise 2: Simple Webserver (6 of 11)

`terraform apply`

- Check Public IPv4 Address is browser.



Exercise 2: Simple Webserver (7 of 11)

```
# Security group definition
resource "aws_security_group" "webserver" {
  name           = "webserver"
  description    = "Allows HTTP traffic"

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

terraform apply

- Try to connect via EC2 Instance
Connect – fails!
- Update main.tf to include port 22 for ingress
- Run apply again and confirm changes

Terraform will perform the following actions:

```
# aws_security_group.webserver will be updated in-place
~ resource "aws_security_group" "webserver" {
  id            = "sg-012d88d4e0c5d00ea"
  ~ ingress     = [
    + {
      + cidr_blocks = [
        + "0.0.0.0/0",
      ]
      + description = ""
      + from_port   = 22
      + ipv6_cidr_blocks = []
      + prefix_list_ids = []
      + protocol    = "tcp"
      + security_groups = []
      + self        = false
      + to_port     = 22
    },
    - {
      - cidr_blocks = [
        - "0.0.0.0/0",
      ]
      - description = ""
      - from_port   = 80
      - ipv6_cidr_blocks = []
      - prefix_list_ids = []
      - protocol    = "tcp"
      - security_groups = []
      - self        = false
      - to_port     = 80
    },
    + {
      + cidr_blocks = [
        + "0.0.0.0/0",
      ]
      + description = null
      + from_port   = 80
      + ipv6_cidr_blocks = []
      + prefix_list_ids = []
      + protocol    = "tcp"
      + security_groups = []
      + self        = false
      + to_port     = 80
    },
  ],
  name      = "webserver"
  tags     = {}
  # (7 unchanged attributes hidden)
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_security_group.webserver: Modifying... [id=sg-012d88d4e0c5d00ea]
aws_security_group.webserver: Modifications complete after 1s [id=sg-012d88d4e0c5d00ea]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Exercise 2: Simple Webserver (8 of 11)

terraform apply

- Try to connect via EC2 Instance Connect – fails!
- Update main.tf to include port 22 for ingress
- Run apply again and confirm changes
- You can see the old ingress rule is dropped and replaced with rules for 80 & 22. Its not required to destroy and apply again, changes like this are merely applied

Exercise 2: Simple Webserver (9 of 11)

- Before

Inbound rules (1/1)							
Filter security group rules							
☒	Name	Security group rule...	IP version	Type	Protocol	Port range	
☒	-	sgr-080743ffa8535c36d	IPv4	HTTP	TCP	80	

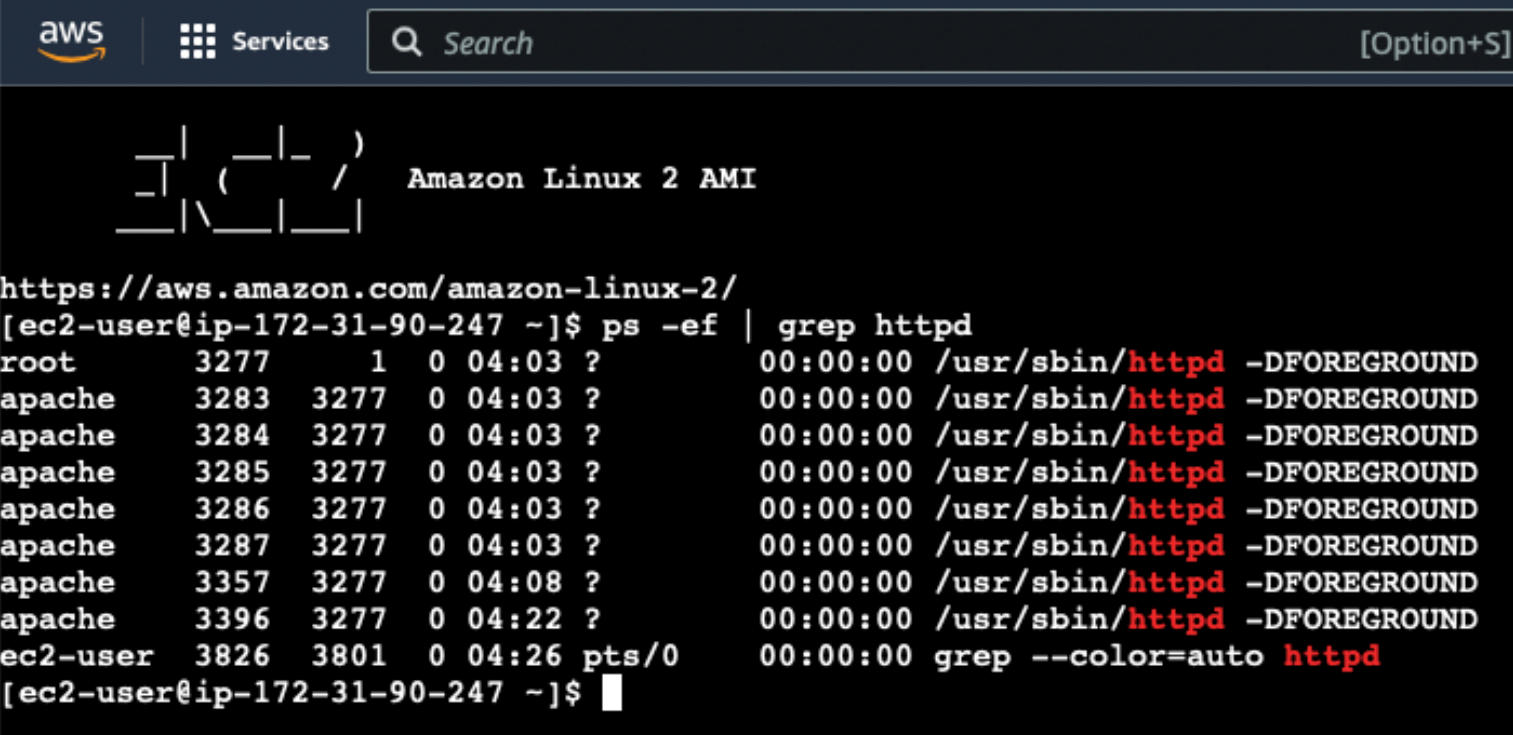
- After

Inbound rules (2)							
Filter security group rules							
☐	Name	Security group rule...	IP version	Type	Protocol	Port range	
☐	-	sgr-080743ffa8535c36d	IPv4	HTTP	TCP	80	
☐	-	sgr-074ba08f7c2a95560	IPv4	SSH	TCP	22	

Exercise 2: Simple Webserver (10 of 11)

`terraform apply`

- EC2 Connect is now working. Also confirmed that Apache homepage still accessible at <https://18.212.18.105>



```
aws | Services | Search [Option+S]

Amazon Linux 2 AMI

https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-172-31-90-247 ~]$ ps -ef | grep httpd
root      3277      1   0  04:03 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3283    3277   0  04:03 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3284    3277   0  04:03 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3285    3277   0  04:03 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3286    3277   0  04:03 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3287    3277   0  04:03 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3357    3277   0  04:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    3396    3277   0  04:22 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
ec2-user  3826    3801   0  04:26 pts/0    00:00:00 grep --color=auto httpd
[ec2-user@ip-172-31-90-247 ~]$
```

Exercise 2: Simple Webserver (11 of 11)

terraform destroy

- Within the scope of the current terraform project, all resources in this terraform scope will be terminated.

```
Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_instance.webserver: Destroying... [id=i-0ffb4b47bd571706a]
aws_instance.webserver: Still destroying... [id=i-0ffb4b47bd571706a, 10s elapsed]
aws_instance.webserver: Still destroying... [id=i-0ffb4b47bd571706a, 20s elapsed]
aws_instance.webserver: Still destroying... [id=i-0ffb4b47bd571706a, 30s elapsed]
aws_instance.webserver: Destruction complete after 30s
aws_security_group.webserver: Destroying... [id=sg-012d88d4e0c5d00ea]
aws_security_group.webserver: Destruction complete after 1s

Destroy complete! Resources: 2 destroyed.
```

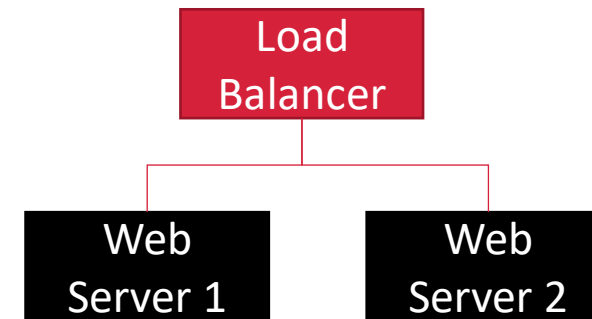
Break (15 Minutes)



Exercise 3: Load Balanced Webserver

Code Overview

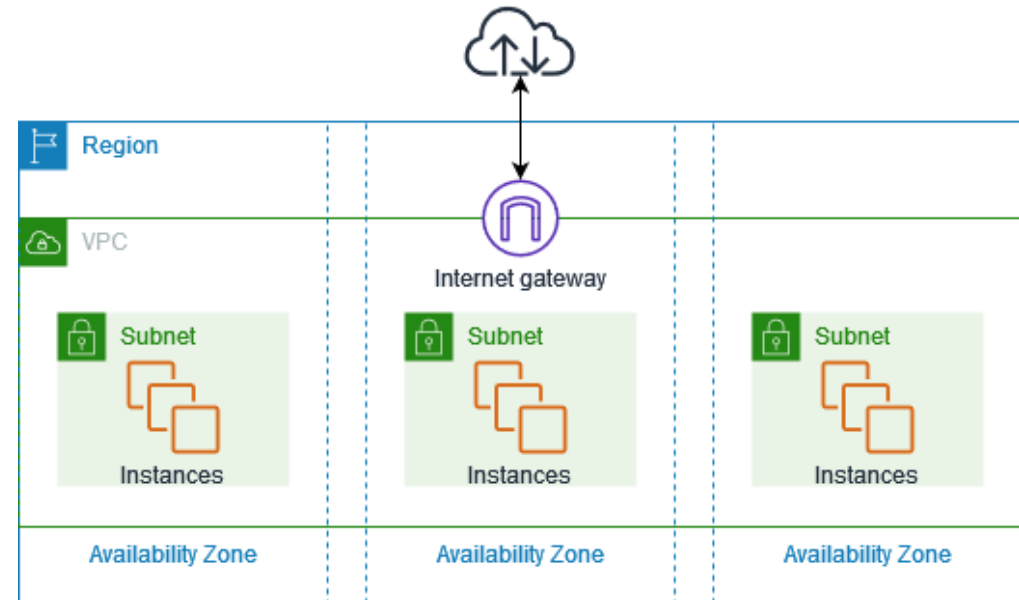
- In this case we are going to build on Exercise 2 by adding a Load Balancer
- In so doing, we will also start working with the important concepts of variables and outputs
- The schematic of this Exercise is shown here...



Exercise 4: Create new VPC with subnets in different AZs

Code Overview

- In this case we are going to build on Exercise 3 by creating a new VPC and a subnet in two different availability zones, so that an application load balancer can be created
- This will be completed without creating any EC2 to demonstrate how these dependent entities can be created effectively
- The schematic of this Exercise is shown here, although in this example we use just 2 subnets



<https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>

Exercise 4: Create new VPC with subnets in different AZs

- Once you have run Terraform apply, it is time to verify the output in the AWS Console.
- Confirm as per your code, that port 80 is open.

Outputs:

```
sg_alb = "terraform-example-alb"
subnet1_id = "subnet-082ec6e6341241faa"
subnet2_id = "subnet-04003c1e2df5386f1"
vpc_id = "vpc-08e1c83a554e96e90"
```

EC2 Global View
Events

- Instances
- Images
- Elastic Block Store
- ▼ Network & Security
 - Security Groups
 - Elastic IPs
 - Placement Groups
 - Key Pairs
 - Network Interfaces
- Load Balancing
- Auto Scaling

sg-0009babadad300b73 - terraform-example-alb Actions ▼

Details

Security group name terraform-example-alb	Security group ID sg-0009babadad300b73	Description Managed by Terraform	VPC ID vpc-08e1c83a554e96e90
Owner 799204530653	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

Inbound rules | Outbound rules | Tags

Inbound rules (1/1) Refresh Manage tags Edit inbound rules

Filter security group rules

☒	Name	Security group rule...	IP version	Type	Protocol	Port range	Source
☒	-	sgr-010acfb86ab1ff906	IPv4	HTTP	TCP	80	0.0.0.0/0

Exercise 4: Create new VPC with subnets in different AZs

- Confirm the VPC and subnet IDs for the Application Load Balancer that you have created.

Outputs:

```
sg_alb = "terraform-example-alb"  
subnet1_id = "subnet-082ec6e6341241faa"  
subnet2_id = "subnet-04003c1e2df5386f1"  
vpc_id = "vpc-08e1c83a554e96e90"
```

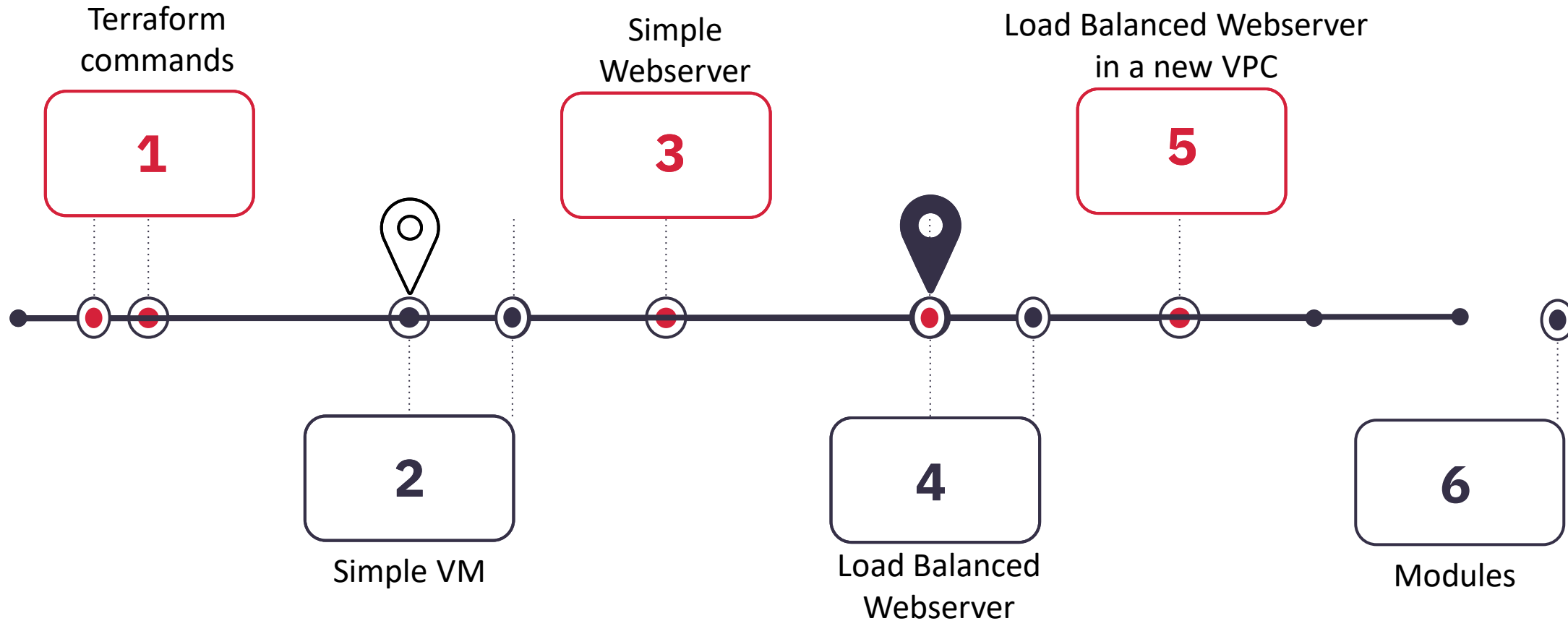
The screenshot shows the AWS Management Console interface for an Application Load Balancer. On the left is a navigation menu with options like EC2 Global View, Events, Instances, Images, Elastic Block Store, Network & Security, Load Balancing (selected), Load Balancers (selected), Target Groups, and Auto Scaling. The main panel displays the details for the 'terraform-asg-example' load balancer. It includes a 'Details' section with a table of attributes: Load balancer type (Application), Status (Active), VPC (vpc-08e1c83a554e96e90), IP address type (IPv4), Scheme (Internet-facing), Hosted zone (Z35SXDTRQ7X7K), Availability Zones (subnet-082ec6e6341241faa in us-east-1a and subnet-04003c1e2df5386f1 in us-east-1b), and Date created (October 17, 2023, 21:21 UTC-04:00). At the bottom, it shows the Load balancer ARN and the DNS name (terraform-asg-example-2043921441.us-east-1.elb.amazonaws.com).

terraform-asg-example			
▼ Details			
Load balancer type Application	Status Active	VPC vpc-08e1c83a554e96e90	IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDTRQ7X7K	Availability Zones subnet-082ec6e6341241faa (us-east-1a) subnet-04003c1e2df5386f1 (us-east-1b)	Date created October 17, 2023, 21:21 (UTC-04:00)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:799204530653:loadbalancer/app/terraform-asg-example/7b48474c9508585a		DNS name terraform-asg-example-2043921441.us-east-1.elb.amazonaws.com (A Record)	

Next steps

- At this point you have reviewed creation of infrastructure, with increasing levels of complexity.
- In the next class, we will pivot to look at some of the features that underpin a robust Terraform practice, such as how to work effectively as part of a bigger team on the same code and infrastructure.
- There is one more code exercise on the infrastructure creation side, which builds further on what you have done here. This code walkthrough will be presented by a very experienced Terraform industry expert as part of class 11.
- In the meantime, we have lots more Terraform to work through as part of the Ansible and Jenkins classes.

Journey Map



Wrap up

- Q & A
- What's 1 thing you learned?
- What's 1 challenge?

References

- Images [Slide 1] used under license from Microsoft PowerPoint stock images.
- Image [Slide 30] 2020 "Woman Sitting on Bench Looking at Mountains" by Yan Krukau used under free license from Pexels.